

CinéConnect

CinéConnect is a full-stack **movie discovery and social platform**. People sign in, explore films powered by [The Movie Database \(TMDB\)](#), curate **favorites** and **watchlists**, **rate** titles, discuss them in **comments** and **live chat rooms**, **follow** other members, and get **in-app notifications**. User-specific data lives in **PostgreSQL**; posters, metadata, and search results are fetched from TMDB at request time (movies are identified everywhere by TMDB numeric ids: `externalMovieId` in the schema).

The codebase is a **pnpm workspace** orchestrated with **Turborepo**: a React SPA ([apps/frontend](#)), an Express API ([apps/backend](#)), and a small shared TypeScript package ([packages/shared](#)) for API response shapes and constants.

What the app does (feature map)

Area	Behavior
Discovery	Trending (public), search / discover / top-rated (authenticated reads), movie detail pages with optional “favorite / on watchlist” when logged in.
Personal lists	Favorites and watchlists are per-user; CRUD via REST.
Engagement	Star ratings (per user per movie), threaded-style comments on movies (create, edit, delete for own comments).
Social	Follow / unfollow users; list followers and following; user search.
Profiles	Display name, email (session), bio, location, favorite genre; avatar upload (stored under <code>uploads/</code> , served by the API).
Auth	Register, login, refresh (rotating refresh tokens, hashed in DB), logout; forgot-password and reset-password when email is configured.
Notifications	Server-created notifications with optional deep-link hints (<code>linkType</code> , <code>targetId</code>); mark read / mark all read.
Chat	Socket.io rooms (e.g. <code>global</code> , <code>film:<tmdbId></code>); messages persisted and paginated via REST for history.

The **marketing-style landing** and **glass UI** components live in the frontend; see [docs/](#) and [apps/frontend](#) for UI-related notes.

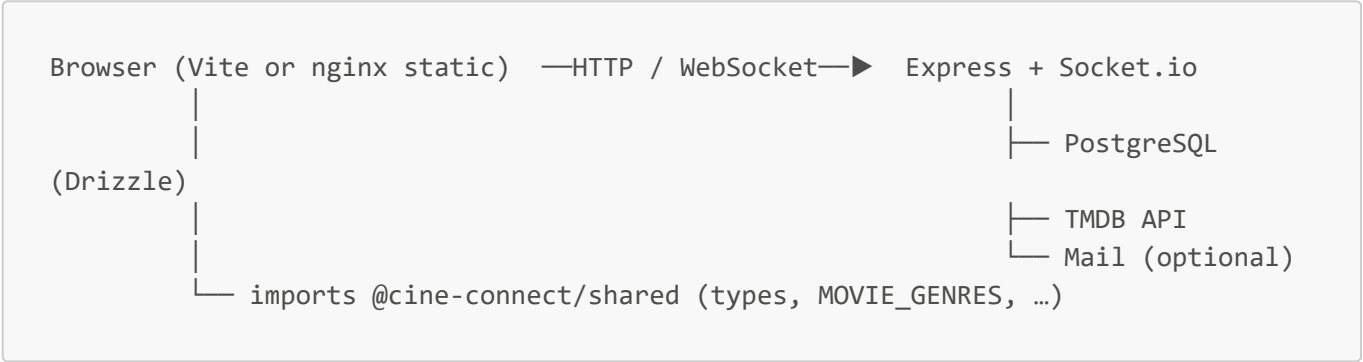
Architecture

Layer	Stack
Frontend	React 19, Vite 7, TanStack Router (file-based routes) & TanStack Query, Tailwind CSS 4, Zustand, Socket.io client, Framer Motion / Motion

Layer	Stack
Backend	Express 4, TypeScript, Drizzle ORM + <code>pg</code> , JWT access tokens + httpOnly cookies (<code>cc_access</code> , <code>cc_refresh</code>), Socket.io, Zod validation, Helmet, rate limiting on sensitive routes
Integrations	TMDB (HTTP) for catalog; Mailgun or SMTP for password-reset emails (optional)
Tooling	pnpm 10, Turbo 2 (<code>build</code> , <code>dev</code> , <code>lint</code> , <code>typecheck</code> , <code>test</code>), Vitest + Supertest (backend), ESLint (frontend), GitHub Actions

Response envelope: JSON under `/api/v1/*` uses `{ success: true, data }` or `{ success: false, error, errors? }` (see `@cine-connect/shared`).

Docs in the browser: OpenAPI is exposed as `/openapi.json`; Swagger UI is served at `/docs`, `/swagger`, and `/?docs=1` (CSP allows the Swagger CDN in development—see `apps/backend/README.md`).



Repository layout

```
cine-connect/
├── apps/
│   ├── frontend/
│   │   └── src/
│   │       ├── routes/           # TanStack Router: index, search, movie.$movieId,
│   │       │   profile*, chat, ...
│   │       ├── components/       # layout, glass UI, landing, shared UI
│   │       ├── service/         # API clients (auth, movies, user, ...)
│   │       ├── hooks/, stores/  # React Query hooks, Zustand (e.g. auth)
│   │       └── lib/             # api-client, socket, route guards, adapters
│   │           └── ...
│   └── backend/
│       └── src/
│           ├── routes/           # Express routers mounted under /api/v1
│           ├── controllers/, services/, schemas/, middlewares/
│           ├── db/              # Drizzle schema, migrations, seed
│           └── socket.ts        # Socket.io server
│               └── ...
├── packages/
│   └── shared/                  # Built to dist/; ApiResponse, User, Movie,
│       MOVIE_GENRES, ...
└── docs/                       # Workflow, planning, UI notes
```

```
|— FullDoc/                                # Extra reference (e.g. schema commentary)
|— docker-compose.yml
|— turbo.json                             # Pipeline tasks (dev is persistent, uncached)
|— cine.ts                                # Repo CLI (git/db/component helpers)
|— p, p.cmd                               # Wrapper entrypoints for ./p
|— .github/workflows/                     # CI (typecheck + backend tests)
```

Package-specific setup: `apps/frontend/README.md` (Vite proxy, Docker + Vite env),
`apps/backend/README.md` (security headers, cookies, uploads).

Database schema (Drizzle / PostgreSQL)

Defined in `apps/backend/src/db/schema.ts`. Migrations are managed with Drizzle Kit (`db:generate`, `db:migrate`, `db:push`).

Table	Purpose
<code>users</code>	Core account: email, password hash, name, timestamps.
<code>refresh_tokens</code>	Hashed opaque refresh tokens; rotation on refresh; cascade on user delete.
<code>password_reset_tokens</code>	One-time hashed tokens for password reset flow.
<code>profiles</code>	1:1 with user: bio, <code>avatar_url</code> , location, favorite genre.
<code>favorites</code>	User ↔ TMDB movie id.
<code>watchlists</code>	User ↔ TMDB movie id.
<code>ratings</code>	User + TMDB movie id + score (unique per user/movie).
<code>comments</code>	User-authored text per TMDB movie id.
<code>follows</code>	Composite PK (follower, following).
<code>notifications</code>	In-app messages; optional <code>link_type</code> / <code>target_id</code> for navigation.
<code>messages</code>	Chat history: <code>room_id</code> , <code>sender_id</code> , <code>content</code> , indexed for room + time pagination.

Authentication and API access

- **Browsers:** After login/register/refresh, the API sets **httpOnly cookies** for access and refresh tokens. The frontend uses `credentials: 'include'` on fetch (see `apps/frontend/src/lib/api-client.ts`). Bearer JWT in the `Authorization` header is still supported (tests, API clients, Socket handshake).
- **CORS:** If `FRONTEND_ORIGIN` or `CORS_ORIGINS` is set, only those origins are allowed (with credentials). In production, missing allowlist means **no** CORS origins—configure explicitly.
- **Cross-site cookies:** If the SPA and API are on different sites, you may need `AUTH_COOKIE_SAME_SITE=none` and HTTPS (**Secure** cookies in production)—documented in `apps/backend/.env.example`.

- **Rate limiting:** Stricter limits apply to `/api/v1/auth/*` (and forgot-password); general API rate limiting wraps all `/api/v1` traffic—see `apps/backend/src/middlewares/rateLimit.middleware.ts`.

HTTP API (`/api/v1`)

All paths below are prefixed with `/api/v1`.

Auth — `/auth`

Method	Path	Notes
POST	<code>/register</code>	Creates user; sets cookies.
POST	<code>/login</code>	Sets cookies.
POST	<code>/refresh</code>	Uses <code>cc_refresh</code> ; rotates refresh token.
POST	<code>/logout</code>	Clears cookies.
POST	<code>/forgot-password</code>	Rate limited; always returns a generic message.
POST	<code>/reset-password</code>	Consumes one-time token from email.

Users — `/users`

Method	Path	Notes
GET	<code>/me</code>	Auth: current user + profile.
PUT	<code>/me</code>	Auth: update profile fields.
POST	<code>/me/avatar</code>	Auth: multipart avatar upload.
DELETE	<code>/me</code>	Auth: delete account.
GET	<code>/search</code>	User search (optional auth).
GET	<code>/:userId</code>	Public profile (optional auth).
GET	<code>/:userId/profile</code>	Same family as above (explicit profile path).
GET	<code>/:userId/followers</code>	List followers.
GET	<code>/:userId/following</code>	List following.

Follows — `/follows`

Method	Path	Notes
POST	<code>/</code>	Body: <code>followingId</code> .
DELETE	<code>/:userId</code>	Unfollow target user.

Movies — `/movies`

Method	Path	Notes
GET	/trending	Public; TMDB trending.
GET	/top-rated	Auth; TMDB top rated.
GET	/discover	Auth; genre + pagination.
GET	/search	Auth; query + optional genre/page.
GET	/rating/:movieId	Aggregate + optional user rating.
POST	/rate	Auth; upsert rating.
GET	/comments/:movieId	List comments for movie.
GET	/:movieId	Detail; optional auth enriches favorite/watchlist flags.
POST	/favorite	Auth; toggle favorite.
GET	/favorites/:userId	Auth; that user's favorites.
POST	/watchlist	Auth; add to watchlist.
GET	/watchlist/:userId	Auth.
DELETE	/watchlist/:movieId	Auth.
POST	/comments	Auth; add comment.
PUT	/comments/:commentId	Auth; update own comment.
DELETE	/comments/:commentId	Auth; delete own comment.

Messages — /messages

Method	Path	Notes
GET	/	Auth; query room, limit, offset — paginated history.

Notifications — /notifications

Method	Path	Notes
GET	/	Auth; list (query params per schema).
PATCH	/read	Auth; mark all read.
PATCH	/:id/read	Auth; mark one read.

Other HTTP endpoints

Path	Purpose
GET /health	Readiness: 200 if DB SELECT 1 succeeds, else 503.
GET /uploads/...	Static uploads + dedicated avatar handler.

Real-time chat (Socket.io)

- **URL:** Same host/port as the HTTP API, path `/socket.io` (the Vite dev server proxies this when using the recommended setup—see frontend README).
- **Handshake auth:** Optional JWT via httpOnly access cookie, `auth.token`, or `Authorization: Bearer ...`. Anonymous connections are allowed; `userId` is then unset.
- **Client** → **server:** `join_room`, `leave_room`, `message { roomId, text }`.
- **Server** → **client:** `message` (broadcast + DB persist), `message_history` on join, `user_joined`, `user_left`.

Room ids are strings, commonly `global` or `film:<tmdbId>`. Full behavior is documented in `apps/backend/src/socket.ts`.

Frontend routes (TanStack Router)

File-based routes under `apps/frontend/src/routes/` include:

Route area	Files (representative)
Shell / errors	<code>__root.tsx</code> , <code>-RootLayout.tsx</code> , <code>-Error404.tsx</code>
Public	<code>index.tsx</code> (landing), <code>login.tsx</code> , <code>register.tsx</code> , <code>forgot-password.tsx</code> , <code>reset-password.tsx</code>
Movies	<code>search.tsx</code> , <code>movie.\$movieId.tsx</code>
User / social	<code>profile.tsx</code> , <code>profile.index.tsx</code> , <code>profile.\$userId.tsx</code> , <code>users.tsx</code> , <code>community.tsx</code> , <code>settings.tsx</code>
Library	<code>favorites.tsx</code> , <code>watchlist.tsx</code>
Activity	<code>notifications.tsx</code> , <code>chat.tsx</code>
Dev	<code>dev.glass.tsx</code> (glass UI playground)

`pnpm build` / `pnpm typecheck` in the frontend runs `tsr generate` first so `routeTree.gen.ts` stays in sync.

Package @cine-connect/shared

- **Name:** `@cine-connect/shared`
 - **Role:** Shared TypeScript types (`ApiResponse`, `User`, `Movie`, `Comment`, request DTOs) and `MOVIE_GENRES` (ids aligned with TMDB).
 - **Build:** `pnpm turbo run build --filter=@cine-connect/shared` (CI builds this before backend tests).
-

Prerequisites

- **Node.js** 22 matches CI; LTS is fine locally.
- **pnpm** — version pinned via **packageManager** in root **package.json**.
- **PostgreSQL** 15+ (Compose uses 15; CI uses 16).
- **TMDB API key** — required for movie routes in production and for CI backend tests.

Quick setup

1. Install

```
pnpm install
```

2. Environment files

File	When to use
Root .env	Copy from .env.example for Docker Compose (Postgres passwords, JWT_SECRET , TMDB_API_KEY , FRONTEND_* , VITE_* for image build).
apps/backend/.env	Local pnpm dev for the API — copy from apps/backend/.env.example .
apps/frontend/.env	Local Vite — copy from apps/frontend/.env.example (proxy vs cross-origin API).

Critical: **JWT_SECRET** is always required for the backend (no fallback). In **NODE_ENV=production** it must be **at least 32 characters**.

TMDB: Required for production movie features; locally the server may start without it but TMDB-backed routes will fail until set.

3. Database

Start Postgres, for example:

```
./p db # macOS/Linux/Git Bash (chmod +x p once)
pnpm exec tsx cine.ts db
docker compose up -d db
```

Apply schema to your database (pick one workflow):

```
pnpm --filter backend db:push # push schema (common in dev / CI)
# or: pnpm --filter backend db:migrate # when using generated migrations
```

Seed demo data:

```
pnpm --filter backend db:seed
```

The seed loads **apps/backend/.env** from the backend package directory, not the shell cwd.

Drizzle Kit shortcuts (from **apps/backend**): **db:generate**, **db:migrate**, **db:push**, **db:studio**.

4. Run

```
pnpm dev           # Turbo: frontend (Vite) + backend (tsx)
pnpm build         # Build shared → apps
pnpm lint
pnpm test          # Backend Vitest suite
pnpm typecheck     # All packages – same as CI typecheck job
pnpm pipeline      # turbo: build + typecheck + test + lint
```

Docker (full stack):

```
pnpm docker:up      # docker compose up -d --build
pnpm docker:up:quick # reuse images, no rebuild
```

Compose services: **db** (5432), **cine-db-test** (5433), **backend** (3000), **frontend** (80/443). Backend depends on healthy **db**; frontend depends on healthy backend.

Environment variables (reference)

Root **.env** (Docker Compose)

Variable	Role
POSTGRES_PASSWORD, POSTGRES_TEST_PASSWORD	Required for Compose DB services.
JWT_SECRET, TMDB_API_KEY	Injected into backend container.
FRONTEND_ORIGIN, FRONTEND_APP_URL, CORS_ORIGINS	CORS and email links.
Mailgun / SMTP_*	Optional transactional email.
VITE_*, VITE_SAME_ORIGIN_API, NGINX_API_UPSTREAM	Build args for the frontend image (baked at build time).

Backend **apps/backend/.env**

See **apps/backend/.env.example**: **PORT**, **DATABASE_URL**, **JWT_SECRET**, **TMDB_API_KEY**, **CORS/cookie** overrides, **FRONTEND_APP_URL**, **Mailgun/SMTP**.

Frontend `apps/frontend/.env`

See `apps/frontend/.env.example`: `VITE_API_BASE_URL` (empty = same-origin + proxy in dev), `VITE_API_PROXY_TARGET`, `VITE_APP_NAME`, `VITE_TMDB_IMAGE_BASE_URL`, production/Docker same-origin patterns.

Testing and CI

- **Unit / API tests:** `pnpm --filter backend test` (Vitest, `supertest`). Requires DB schema (`db:push` in CI) and env including `JWT_SECRET`, `DATABASE_URL`, `TMDB_API_KEY`.
- **Workflow:** `.github/workflows/tests.yml`
 - Job **typecheck**: `pnpm install` → `pnpm typecheck` (Node 22).
 - Job **backend-tests**: Postgres service container → build shared → `pnpm --filter backend db:push` → `pnpm --filter backend test` with `TMDB_API_KEY` from **repository secret** `TMDB_API_KEY`.
- **Coverage:** `pnpm --filter backend test:coverage` locally.

GitHub Actions configuration

Path: **Settings** → **Secrets and variables** → **Actions**.

Kind	Name	Purpose
Secret	<code>TMDB_API_KEY</code>	Required for backend test job (validated in workflow).
Variable	<code>VITE_API_BASE_URL</code>	Optional; if your workflow builds the frontend.
Variable	<code>VITE_APP_NAME</code>	Optional branding in builds.
Variable	<code>VITE_TMDB_IMAGE_BASE_URL</code>	Optional TMDB CDN base.

Docker notes

- **Frontend image:** Static nginx. **`VITE_*` are compile-time**—change `.env` and **rebuild** the frontend image. Same-origin API mode proxies `/api` to `NGINX_API_UPSTREAM` (e.g. `http://backend:3000` in Compose).
- **Healthchecks:** Postgres `pg_isready`; backend Node `fetch` to `/health`; frontend `wget` to port 80. `HEALTHCHECK` in Dockerfiles mirrors this for `docker run`.
- **Uploads:** Backend mounts `./apps/backend/uploads` so avatars survive container restarts.

CLI shortcut (`./p`)

Cross-platform wrappers (`p`, `p.cmd`) forward to `pnpm` scripts in root `package.json`:

- **macOS/Linux/Git Bash:** `chmod +x p` once, then `./p <command>`.
- **Windows PowerShell:** `.\p <command>`.
- **Windows CMD:** `p <command>`.

Command	Description
<code>p gac</code>	Git add all + commit (interactive/manual flow).
<code>p gp</code>	Push and watch pipeline.
<code>p cb</code>	Interactive branch switch.
<code>p gt</code>	Show current branch.
<code>p db</code>	Start Docker database.
<code>p gen component <Name></code>	Scaffold a React component.
<code>p gg</code>	Interactive menu.
<code>pnpm typecheck</code>	Monorepo TypeScript (CI parity).

Turborepo tasks (`turbo.json`)

Task	Behavior
<code>build</code>	Depends on upstream <code>^build</code> ; outputs <code>dist/**</code> .
<code>dev</code>	Persistent, not cached.
<code>lint, typecheck</code>	Per-package scripts.
<code>test</code>	Depends on <code>^build</code> (shared must build first for backend tests).

CinéConnect splits the SPA and API so you can deploy and scale them separately, keep cookies and CORS explicit, and evolve the contract through `@cine-connect/shared` and OpenAPI in lockstep.